

Sumate a **RUgiar**. Unamos nuestro conocimiento y energía.

Un fork de `unitree_rl_gym` que entrena, transforma y maneja policies de RL para humanoides Unitree — con una base sólida ya funcionando y siete frentes abiertos esperando a la persona indicada. Sí, vos, vos que estás leyendo esto.

7 áreas · elegí la tuya

Ramas cortas, PR simple

Arrancás hoy

Lo que hace atractivo este momento para colaborar es que **RUgiar recién está arrancando en serio**. Hay una base sólida — entrenamiento funcionando, control en vivo funcionando, las partes críticas ya cubiertas — pero todas las áreas tienen techo, desde la UI web hasta la integración con captura de movimiento humano. Quien entre ahora tiene margen real para dejar su huella, no para retocar los bordes de algo ya cerrado.

Este documento es la puerta de entrada, no el manual completo. Profundidad cuando la necesites: el material didáctico completo, la arquitectura entera, y el skill `rugiar` para el uso diario del CLI y el driver.

Elegí una y tomá la posta

El proyecto se divide siguiendo el ciclo de vida de una policy: entrenarla → transformarla → manejar con ella un robot. Pensadas para trabajarse en paralelo — cada una tiene dueño potencial: vos.

§1

Entrenamiento

Mejorar cómo se lanzan y siguen los jobs de RL, local o en Kaggle, y el catálogo de policies resultante.

`legged_gym/control/training.py`

§2

Operaciones sobre policies

Fusión de pesos, destilación / behavior cloning — hacer más policies a partir de las que ya existen.

`fusion.py · distillation.py`

§3

Control

El motor del robot en vivo: cambio de policy, seguridad, transporte, adapters sim/real.

`service.py · transport.py · supervisor.py · safety.py`
 ref:
`examples/joystick_controller.py`

§4

UI Web

El panel de control en el navegador — hoy el área con más superficie visible y más margen de crecimiento.

`web/index.html · web/app.js`

§5

CLI

La herramienta de terminal — entrenar, fusionar, destilar sin abrir un navegador.

`legged_gym/cli/rugiar.py`

§6

Driver del robot

El proceso que arma todo y corre el loop del robot, en simulación o real.

`rugiar_driver.py · rugiar_driver_gaze.py`

§7 · SHIPPED

Motion library — de captura de movimiento a policy entrenada

Lo que esta sección pedía como "terreno sin dueño" ya se construyó y está validado en vivo: un pipeline completo desde un dataset externo de motion capture ([g1-moves](#)) hasta una policy G1 full-body entrenada imitando ese movimiento, con overlay "ghost" de referencia en vivo en viser, selector de clips y Create Policy conscientes de mjlab, training backend propio, y panel de rewards con las 9 variables de tracking de este mundo.

`process_reference_motion_mjlab.py · mjlab_train.py · docs/motion_imitation_integration.md`

FRONTERA TODAVÍA ABIERTA ACÁ

El pipeline de arriba corre sobre Genesis/mjlab, sin depender de NVIDIA. El stack de simuladores NVIDIA (Isaac Lab / Isaac Gym) para este mismo mundo de motion imitation lo está avanzando otro equipo, integración pendiente. Más allá de eso: sumar nuevas fuentes de movimiento — video propio digitalizado, otros datasets de motion capture, captura desde el robot mismo — sigue siendo terreno real para quien se sume. [tu nombre acá](#)

Antes de arrancar: Entrenamiento produce carpetas `policies/<nombre>/` — ese directorio es el contrato con el resto del sistema. Control no se instancia solo, lo arma el Driver. La UI Web solo habla WebSocket contra Control, sin otro camino de entrada. El CLI no toca Control ni el Driver — cero imports, la frontera más limpia del repo.

POR QUÉ ESTÁ ARMADO ASÍ

Simulación y robot real, sin reescribir nada

El sistema depende de un protocolo, no de Genesis ni del SDK de Unitree — por eso pasar de simulación a robot real no debería requerir reescribir nada arriba.

Dos cosas para saber antes de arrancar, no para frenarte: `training.py` hace bastante hoy — si ves una separación natural, proponela. Y `web/app.js` todavía no tiene un registro de paneles — si vas a sumar uno, avisá en el equipo para coordinar con quien esté tocando otro al mismo tiempo.

Política de branching

Lo más laxo posible. Dos reglas duras, el resto es criterio de cada uno.

REGLA DURA

Nadie pisa main directo

Todo cambio entra por rama propia.

REGLA DURA

Todo se mergea por PR

Aunque la revises vos mismo — deja registro de qué cambió y por qué.

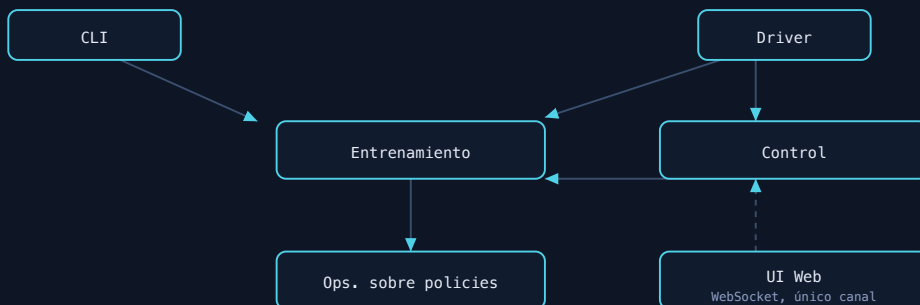
El resto son sugerencias, no obligaciones:

- Usá nombres de rama y mensajes de commit significativos — que cuenten algo del cambio, no `fix` / `wip` / `cambios`. Preferentemente en inglés, pero que se entienda importa más que el idioma.
- Correr los tests ayuda (`pytest tests/`, con `SIMULATOR` según tu entorno), y agregar tests de lo que hagas es aconsejado — no exigido.
- Como norte de diseño tratamos de seguir SOLID, DRY y LEAN — porque permite que varias personas toquen esto en paralelo sin pisarse.
- Si tu cambio cruza dos áreas, avisar en la PR quién coordinó qué ayuda — de nuevo, criterio, no regla.

Punto de partida laxo a propósito. Si el equipo crece o algo empieza a doler, se ajusta.

EL MAPA

Cómo se conectan las siete áreas



CLI y Driver llegan a Entrenamiento por caminos distintos que nunca se cruzan · UI Web sólo habla con Control

Leído en una frase: el Driver es el único que arma Control; Control es la única puerta de entrada de la UI; el CLI y la UI llegan a Entrenamiento por caminos distintos que nunca se cruzan; y todo lo que las áreas comparten en disco son las carpetas `policies/<nombre>/`.

A dónde ir según lo que vayas a tocar

SI VAS A...	EMPEZÁ POR	Y ADEMÁS
Entrenamiento	<code>training.py</code>	<code>result.json</code> / <code>progress.json</code> solo documentados en comentarios — buena primera mejora.
Operaciones sobre policies	<code>fusion.py</code> / <code>distillation.py</code>	Orquestación en <code>training.py</code> — coordiná con quien esté ahí.
Control	ARCHITECTURE.md §3, después <code>service.py</code>	Diagrama de secuencia y nota de concurrencia: lectura obligatoria antes de la primera línea.
UI Web	<code>send()</code> , <code>call()</code> , <code>applyStatus()</code>	Más espacio para crecer, más impacto visible, hoy.
CLI	<code>rugiar.py</code>	Preservá la ausencia de imports de Control/Driver.
Driver del robot	<code>rugiar_driver.py</code>	Cambios en helpers compartidos van también a <code>_gaze.py</code> .
Hardware real	<code>deploy_real/real_adapter.py</code>	Sin probar en robot físico — si tenés acceso a un G1, sos quien puede cerrar esa brecha.
Motion library / mjlab	<code>docs/mjlab_training_contract.md</code>	Frontera abierta: nuevas fuentes de movimiento y el stack NVIDIA, ver §7.

ANTES DE TU PRIMER PR

Instalación, según tu sistema

Tres caminos según de dónde vengas: nativo en Mac/Linux, un solo comando con Docker en cualquier sistema (incluido Windows), o Kaggle si no tenés GPU propia y querés entrenar en la nube.

MACOS / LINUX **NATIVO**

Clonás, armás un entorno virtual de Python 3.12 e instalás las dependencias. Sin GPU no hay problema — en Apple Silicon corre sobre CPU o Metal.

```
# 1. clonar y entrar al repo
git clone https://github.com/josetabuyo/RobotUniversityGiar.git
cd RobotUniversityGiar

# 2. instalador de un comando
./install.sh
# con Kaggle: ./install.sh --with-kaggle

# 3. cada terminal nueva:
source .venv/bin/activate
export SIMULATOR=genesis
```


Kaggle: GPU gratis sin tener una propia

Entrenamiento (§1) puede lanzar jobs de RL en un kernel de Kaggle en vez de tu máquina local. Configuración de una sola vez.

1. Creá una cuenta gratis en kaggle.com.
2. Verificá tu teléfono en kaggle.com/settings → Phone Verification (obligatorio para GPU, ~30h/semana gratis).
3. Creá un token de API en kaggle.com/settings → API → Create New Token (descarga `kaggle.json`).
4. Instalalo localmente:

```
mkdir -p ~/.kaggle
mv ~/Downloads/kaggle.json ~/.kaggle/kaggle.json
chmod 600 ~/.kaggle/kaggle.json
```

5. Instalá el paquete `kaggle` (`pip install -e .[cloud]`) y verificá credenciales.

Los jobs de Kaggle corren siempre sobre Isaac Gym (no Genesis) — GPUs Pascal (P100) no soportan el backend GPU de Genesis. No afecta tus corridas locales.

Token de control: la llave del robot

Cuando el Driver corre con `--real`, el socket de control queda expuesto en la red del robot. Un token es un secreto compartido que vos elegís.

```
# generar
openssl rand -hex 16

# arrancar el driver
python legged_gym/scripts/rugiar_driver.py \
  --policy stable;policies/stable/checkpoint.pt \
  --control_port 9013 --real --token <tu-secreto>

# conectarse
ws://<host>:9013/ws?token=<tu-secreto>
```

Sin `--token`, el handshake se acepta de cualquiera en la red del robot. Aceptable solo para simulación local de confianza; nunca para `--real`.

Elegí un área, abrí una rama, y arrancá. Que ruja.

Si algo de este documento quedó viejo apenas empieces — arreglalo vos mismo. Es, literalmente, el primer PR ideal.